Design Decisions

1. Reactive and combined Upvoting

While my old implementations of Fritter had a separation of upvoting and removing that upvote, I wanted to include them into one toggling experience with implementing a heart icon with the coloring of pink versus grey to indicate your current state. I thought this combination of actions was important because it simplified the cognitive load of the user as well as the interface of the individual freet. The limitations of this might be how I implemented it within the code (in regards to the asynchrony of Axios posting), but I think given the scale of A5 as well as how it works so far, it's appropriate!

Alternatives Considered: I considered leaving the two features separate, largely because it would have simplified the code, but ultimately decided to combine them because I wanted to ensure the user experience was prioritized. Having both "Upvote" and "Remove Upvote" is a little bulky, and the separation also introduces friction in actions that might be used again and again (in moving the mouse the little distance to do the complementary action).

2. Hiding features that you can't access

I decided to hide actions like deleting and editing freets on the freets that don't belong to the currently logged in user so that the currently logged in user wouldn't be confused and/or overwhelmed by what set of actions is available to them. By doing so, I was again able to simplify my Freet interface so that users could be more directly concerned with their available actions. By allowing them access to the "Delete Freet" example on a freet that they can't even delete would only introduce unsatisfactory interactions with Fritter. I don't think this implementation has limitations; in fact, by limiting the view, users can focus more on relevant components of Fritter.

Alternatives Considered: I considered everything the same such that all Freets would have the same sets of action available; however, this was ultimately scrapped in order to contribute to a more seamless and straightforward user experience.

3. An unfiltered feed!

The current main feed is very unfiltered and allows access to all freets as long as you're logged in. I chose this with the intention of helping users branch out more with who is currently on Fritter, given the very small network of users there are.

A limitation of this implementation is that there's a little too much noise / it's overwhelming to see all the freets once Fritter has a large number of users. In the future / if I were to think beyond the scope of A5, I would want to either implement filtering with who users are following (as this functionality is not currently pursued significantly) or allow different kinds of filtering.

Alternatives Considered: An alternative considered was the aforementioned filtering by followers. I didn't choose this because of Fritter's current position as a web app, but hope to build on this more as the user base gets more complicated

4. Having separate and individual pages for users

I extended Fritter's functionality by providing routing to users on the platform (beyond your own account). The rationale behind this decision was that I wanted to expand the mental space of Fritter beyond just the standard feed and your own account; this also allows for a degree of filtering to only look at a specific user's freets as well as an separation of freets and users. This was also important in learning how routes function on the frontend (and how to ensure everything is still hooked up to the backend!).

Alternatives Considered: I thought of keeping Fritter as a single route, and while this would have been simpler to implement, I thought it was more of a burden in thinking through how the different features and concepts in Fritter come together. I also wanted to prioritize the learning opportunity of developing the specific pages for each user!

Ethical / Social Reflection

Reflecting on my wireframes made me quickly realize how the wireframes represented a very particular slice of the actual, cohesive Fritter implementation. Because the A4 wireframes only focused on a few concepts, I was surprised by the much larger scope of A5 that had me consider what it meant to be immersed in the application experience. For example, I didn't consider my second design decision of hiding features you can't access and considering how the user navigates through the interface. My interface design thus became a lot more charged in considering the bounds and limitations of the currently logged in user; when wireframing, it's easy to think about the full picture. However, when designing for these experiences, I had to scope down to what should be visible / accessible.

On accessibility, implementing the front end of Fritter quickly made me understand my limitations as a developer with no prominent disabilities. I had to consciously remind myself that my own experience when pretending to be the user and debug the app was fundamentally limited as my own experience, and it was very easy to have tunnel vision with just wanting to make something "work." I had to thus think a lot more about what it means for a feature to "work" – if it only works for part of the user base, can one actually say it works? What do you need to think about when you're working on a much richer app, like Instagram?

Working on A5 within a time-crunch of a stressful week also made me realize that often time, all of your ambitions regarding the implementation of an app don't occur, and certain features and functionalities need to be prioritized. This brings about a lot of ethical implications of within a time crunch, what are we forgoing? What do I prioritize as a developer? Because I care to some degree about the user experience, I went through the process of debugging and implementing state such that we could toggle between upvoting and removing the upvote. While this doesn't seem like it has huge social /ethical implication, the idea of these tradeoffs is key to understanding how the software we interact with on a day to day evolves.